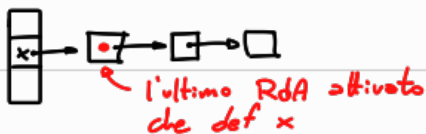


Implementazione scoping dinamico

Algoritmo (shallow access) basato su CRT
utilizza struttura dati centrale

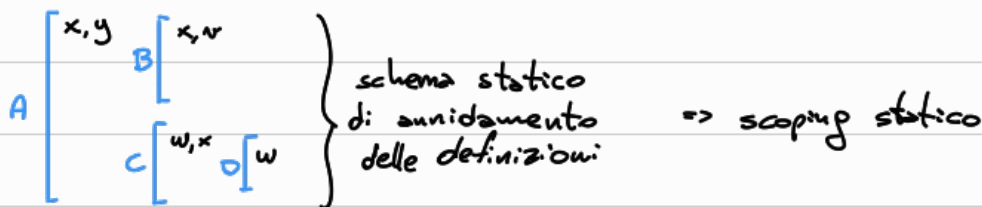
CRT: Tabella centrale dei riferimenti:

- evita ricerche lineari
- mantiene in modo centralizzato le informazioni necessarie per accedere al RdA dinamico corretto
- è una tabella con una voce ^{riga} per ogni identificatore usato nel programma
- ogni identificatore punta ad una lista t.c. in testa si trovano le informazioni dell'ultimo (dinamicamente) RdA che definisce l'identificatore

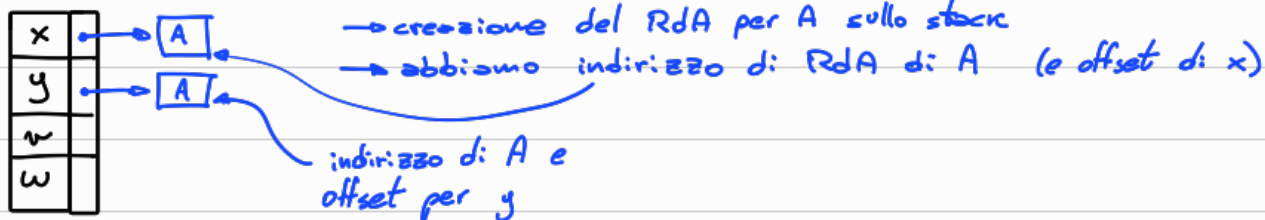


Esempio

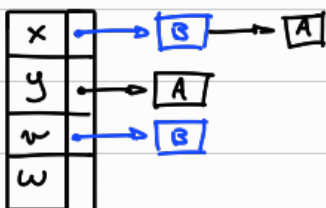
A → B → C → D seq. di chiamate



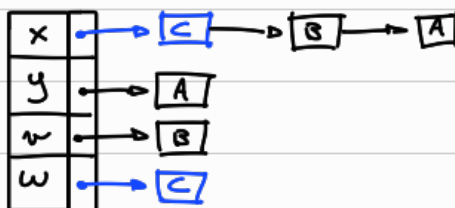
CRT



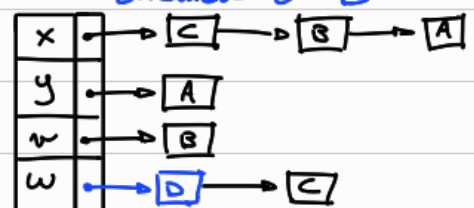
chiamata di B



chiamata di C



chiamata di D

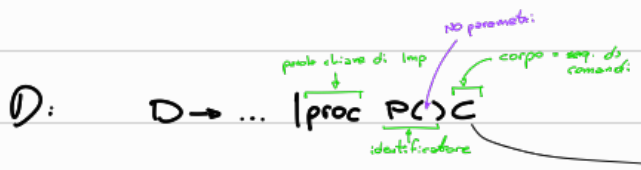


All'uscita da un blocco viene distrutta RdA e viene eliminata la testa di ogni lista linkata ... DA FINIRE

[0:23:00]

Procedure in IMP

- procedure vanno definite ⇒ modifica delle dichiarazioni
- procedure vanno chiamate ⇒ modifica dei comandi



Aggiungiamo agli oggetti denotabili le astrazioni: $\lambda E. C$

Annotations:
- λ : funzione
- E : lista vuota di parametri
- C : corpo

Regola per la dichiarazione

$$\rho \vdash \langle \text{proc } P() C, \sigma \rangle \rightarrow \langle [P \mapsto \lambda E. C'], \sigma \rangle$$

associazione costante di P al suo codice
→ no locazione

$$C' = \begin{cases} p; C & \text{se scoping statico} \\ C & \text{se scoping dinamico} \end{cases}$$

se scoping statico: fisso l'ambiente presente al momento della def di P ovvero alla sua esecuzione
se scoping dinamico: si usa l'ambiente della chiamata di P

C: $C \rightarrow \dots | P()$
Annotations:
- $P()$: identificatore con lista vuota di parametri

Regola per la chiamata

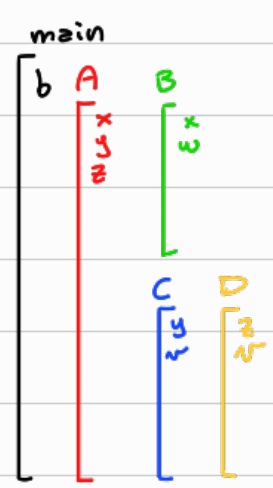
$$\rho \vdash \langle P(), \sigma \rangle \rightarrow \langle C', \sigma \rangle, \quad \rho(P) = \lambda E. C'$$

Annotations:
- $\rho(P)$: $p; C$
- C : C
- $\rho[p]$: environment after binding p to C
- con definizione $\text{proc } P() C$ nell'ambiente ρ

Esercizio su algoritmi di scoping

[0:39:00]

```
{ int b;  
void A() {  
    int x=0, y=0, z=5;  
    void B() {  
        int x=3, w=3;  
        x = y+z;  
    }  
    void C(int a) {  
        int y=1, v=1;  
        void D() {
```



- $Sd(\text{main}) = 0$
- $Sd(A) = 1$
- $Sd(B) = Sd(C) = 2$
- $Sd(D) = 3$

chiamate:

$\text{main} \rightarrow A \rightarrow C \rightarrow D \rightarrow B$

[0:48:41]

IMPORTANTE !!

Concetti da vanno definiti:

- cosa distinguere **scoping statico e dinamico** } def
- quando servono queste regole
- profondità statica (Sd) e spiegare!
- link statico e dinamico
- CRT, cosa è e come si gestisce

```
int z=2, v=2;
```

```
B();
```

```
v=x+y;
```

```
}
```

```
D();
```

```
x=z+y;
```

```
}
```

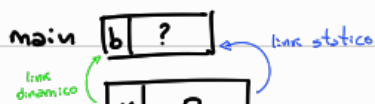
```
C(7);
```

```
}
```

```
A();
```

```
}
```

Scoping statico



A

x	0
y	0
z	5

C

a	7
y	1
v	1

D

z	2
v	2

B

x	3
w	3

chiamata main → A

calcolo link statico:

$$K = Sd(chiamante) - Sd(chiamato) + 1$$

$$\Rightarrow K_A = Sd(main) - Sd(A) + 1 = 0 + 1 - 1 = 0$$

$$CS(A) = indirizzo(main) = main$$

dobbiamo risalire di K
→ la catena statica per
trovare il genitore

chiamata A → C(7)

calcolo link statico:

$$K_C = Sd(A) - Sd(C) + 1 = 1 - 2 + 1 = 0$$

$$CS(C) = ind(A) = A$$

chiamata C → D

calcolo link statico:

$$K_D = Sd(C) - Sd(D) + 1 = 2 - 3 + 1 = 0$$

$$CS(D) = ind(C) = C$$

chiamata D → B

calcolo link statico

$$K_B = Sd(D) - Sd(B) + 1 = 3 - 2 + 1 = 2$$

$$CS(B) = CS(CS(D)) = CS(C) = A$$

Esecuzione di B: $x = y + z$
 locale

$N_a = Sd(\text{Proc che usa } a) - Sd(\text{Proc più vicina def } a)$
 schema dinamico
 dice di quanto risalire la catena statica per risolvere a non locale

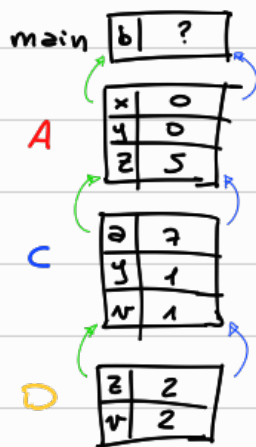
$$N_y = Sd(B) - Sd(A) = 2 - 1 = 1 \quad y = 0$$

$$N_z = Sd(B) - Sd(A) = 2 - 1 = 1 \quad z = 5$$

$$x = 0 + 5 = 5$$

x	5
w	3

Terminazione di B $\rightarrow$ distrugge RbA per B



Esecuzione di D

$$v = x + y$$

locale

$$N_x = Sd(D) - Sd(C) = 3 - 1 = 2 \quad x = 0$$

$$N_y = Sd(D) - Sd(A) = 1$$

$$y = 1$$

$$v = 0 + 1 = 1$$

D

z	2
v	1

Terminazione di D $\rightarrow$ distruzione RbA D

Esecuzione di C

$$x = z + y \rightarrow \text{locale} = 1$$

$$N_x = Sd(C) - Sd(A) = 2 - 1 = 1 \quad x = 0$$

$$N_z = Sd(C) - Sd(A) = 2 - 1 = 1 \quad z = 5$$

$$x = 5 + 1 = 6$$

A

x	6
y	0
z	5

Terminazione di tutte le procedure e distruzione RbA

[1:02:00]

Scoping dinamico

main $\rightarrow$ A

b	$\rightarrow$ main, ?
x	$\rightarrow$ A, 0
y	$\rightarrow$ A, 0
z	$\rightarrow$ A, 5
v	
w	

A $\rightarrow$ C

b	$\rightarrow$ main, ?
x	$\rightarrow$ A, 0
y	$\rightarrow$ C, 1 $\rightarrow$ A, 0
z	$\rightarrow$ A, 5
v	$\rightarrow$ C, 1
w	

C $\rightarrow$ D

b	$\rightarrow$ main, ?
x	$\rightarrow$ A, 0
y	$\rightarrow$ C, 1 $\rightarrow$ A, 0
z	$\rightarrow$ D, 2 $\rightarrow$ A, 5
v	$\rightarrow$ D, 2 $\rightarrow$ C, 1
w	

D → B

Esecuzione di B

b	→ main, ?
x	→ B, <u>3</u> → A, 0
y	→ C, <u>1</u> → A, 0
z	→ D, <u>2</u> → A, 5
v	→ D, 2 → C, 1
w	→ B, 3

$$x = y + z = 3$$

uscita da B

esecuzione di D

$$v = x + y = 1$$

b	→ main, ?
x	→ A, <u>0</u>
y	→ C, <u>1</u> → A, 0
z	→ D, 2 → A, 5
v	→ D, <u>2</u> → C, 1
w	

uscita da D

esecuzione di C

$$x = z + y = 6$$

b	→ main, ?
x	→ A, <u>0</u> ⁶
y	→ C, <u>1</u> → A, 0
z	→ A, <u>5</u>
v	→ C, 1
w	